RETRIEVAL-AUGMENTED GENERATION (RAG)

Five-page study notes • Foundations to practical implementation

1. What is RAG?

Retrieval-Augmented Generation (RAG) is an architecture that combines information retrieval with a Large Language Model (LLM). Before generating an answer, the application searches an external knowledge source, retrieves relevant passages, and supplies them to the LLM as context. The model then generates a response grounded in that evidence. In simple terms: **retrieve first, generate second.**

Why is RAG useful?

An LLM's built-in knowledge may be incomplete, outdated, or unrelated to an organisation's private data. It may also produce fluent but unsupported statements, known as hallucinations. RAG connects the model to information that can be updated independently of the model itself.

- **Fresh knowledge:** update documents and indexes without retraining the LLM.
- **Private knowledge:** answer questions from company PDFs, manuals, policies, tickets, and internal documentation.
- **Grounded answers:** provide source names, pages, or passages so users can verify claims.
- **Domain context:** make specialised information available at question time.
- **Flexible maintenance:** replace or remove indexed content as knowledge changes.

2. RAG vs. fine-tuning vs. long context

Method	Best for	Limitation
RAG	Changing facts, private documents, citations, external knowledge	Depends on parsing and retrieval quality
Fine-tuning	Changing behaviour, style, task patterns, output format	Not ideal for frequently updated factual content
Long context	A small known set of content that fits in the prompt	Can be costly and include irrelevant information

3. The basic RAG pipeline

- **Ingestion:** load files and extract text plus metadata.
- **Chunking:** split long content into focused, manageable passages.
- **Embedding:** convert each chunk into a numerical vector.
- **Indexing:** store vectors, text, and metadata in a searchable store.
- **Retrieval:** embed the user's query and find relevant chunks.
- **Augmentation:** place retrieved passages into the LLM prompt.
- **Generation:** produce an answer using the question and retrieved evidence.

Key point: RAG is not one model or one database. It is a multi-stage pipeline. Weakness in any stage—extraction, chunking, embeddings, retrieval, prompt design, or generation—can reduce answer quality.

DOCUMENT INGESTION & CHUNKING

4. Document ingestion

Ingestion converts source material into clean, searchable content. Common sources include PDF, DOCX, TXT, Markdown, HTML pages, knowledge-base articles, and database records. A robust ingestion process preserves the document structure and source details needed later for retrieval and citations.

- **Load:** read the file with a format-appropriate loader.
- **Extract:** obtain text, headings, tables, and page information where possible.
- **Clean:** remove repeated headers, footers, broken whitespace, and extraction artefacts without changing meaning.
- **Attach metadata:** document title, source path, page, section, version, date, tenant, and permissions.
- **Validate:** detect empty files, duplicates, unsupported formats, and extraction errors.

5. What is chunking?

Chunking divides a long document into smaller passages called **chunks**. Search is normally performed over these passages instead of an entire book or PDF. A useful chunk is focused enough to retrieve precisely but contains enough context to be understood independently. For example, an HR handbook can be split into passages so a question about annual leave retrieves the leave-policy section, not the whole handbook.

6. Chunk size and overlap

Term	Meaning	Trade-off
Chunk size	Approximate length of each passage, often measured in tokens.	Small chunks are precise but may lose context; large chunks may contain noise.
Overlap	Text repeated between neighbouring chunks.	Preserves ideas across boundaries but adds storage and duplicate results.
Separators	Boundaries such as headings, paragraphs, or sentences.	Structure-aware splitting is usually better than arbitrary cuts.

7. Common chunking strategies

- **Fixed-size:** split at a chosen token or character count. Simple, but may break sentences.
- **Recursive:** try headings, paragraphs, and sentences in order; split further only when required.
- **Structure-aware:** use page layout, headings, sections, and table boundaries. Useful for reports and manuals.
- **Semantic:** detect topic changes and group text by meaning. Can improve coherence but is more complex.

8. Metadata and document identity

Metadata describes a chunk. A chunk might contain the policy text and metadata such as *document=HR_Policy.pdf*, *page=7*, *section=Leave*, and *version=2026*. Preserve metadata through indexing and retrieval; it enables filtering, access control, and citations.

Practical advice: begin with a reasonable chunk size and overlap, inspect the resulting chunks manually, then test alternative settings with real user questions. There is no universally best chunk size.

EMBEDDINGS, VECTOR DATABASES & SEARCH

9. What are embeddings?

An embedding model converts text into a list of numbers called a **vector**. The vector represents semantic features of the text. Passages with related meanings often have vectors that are close in the embedding space, even if their wording differs. For example, “reset my password” and “I forgot my login credentials” may be semantically similar.

10. Similarity search

At query time, the user's question is converted into a vector using an embedding model. The vector store compares it with stored chunk vectors and returns nearby matches. Common similarity measures include cosine similarity, dot product, and Euclidean distance. The appropriate metric depends on the model and vector normalisation. Always check whether a database's returned score means higher-is-better or lower-is-better.

11. What is a vector database?

A vector database stores embeddings and supports efficient nearest-neighbour search. A RAG record commonly contains the chunk text, its embedding, and metadata. Depending on the product, it may also support metadata filters, hybrid search, collections or namespaces, and update/delete operations.

Technology	Typical use
FAISS	Local similarity-search library, useful for experiments and in-process indexes.
Chroma	Developer-friendly vector store for prototypes and local applications.
Qdrant	Vector search with metadata filtering and production-oriented capabilities.
Pinecone	Managed vector database service for hosted vector search.
pgvector	PostgreSQL extension for vector storage and similarity search.

12. Dense, sparse, and hybrid retrieval

- **Dense retrieval:** uses semantic embeddings. Helpful when the query and document use different words.
- **Sparse retrieval:** uses lexical methods such as BM25. Strong for exact terms, names, IDs, and rare keywords.
- **Hybrid retrieval:** combines dense and sparse results to benefit from semantic and keyword matching.

13. Top-K and metadata filters

Top-K is the number of candidate chunks returned by a search. A low K may miss useful evidence; a high K may add irrelevant context and consume the LLM's context window. Metadata filters narrow search to a document, date, department, user, or permitted collection. Choose K and filters by testing—not guesswork.

14. Embedding consistency

Use a compatible embedding model for indexing documents and embedding user queries. If the embedding model changes, re-embed the documents and rebuild the index. Mixing vectors from incompatible embedding spaces can make similarity results unreliable.

PROMPTING, GENERATION & IMPLEMENTATION

15. How retrieval becomes an answer

After retrieval, the application assembles a prompt containing system instructions, the user's question, and selected context passages. This is the augmentation step. The LLM generates a response using the supplied evidence. A good prompt explains which sources to trust, what to do when evidence is missing, and how to cite sources.

16. Example grounded prompt

Instruction: Answer using only the context below. If the context is insufficient, say so. Do not invent facts. Cite the source identifier for important claims.

Context: [employee_handbook.pdf, page 7] Employees may request annual leave through the HR portal. Requests are subject to manager approval.

Question: How do employees request annual leave?

Grounded answer: Employees submit an annual-leave request through the HR portal, and the request requires manager approval. [employee_handbook.pdf, p. 7]

17. Simplified RAG pseudocode

```
documents = load_documents(files)
chunks = split_into_chunks(documents)
vectors = embed_documents(chunks)
vector_store.add(chunks, vectors, metadata)

question = get_user_question()
q_vector = embed_query(question)
matches = vector_store.search(q_vector, top_k=5)
context = format_context(matches)
prompt = build_prompt(question, context)
answer = llm.generate(prompt)
return answer_with_sources(answer, matches)
```

18. Main components of a beginner project

- **Document loader:** reads PDF/text files and creates document objects.
- **Text splitter:** creates chunks while retaining useful metadata.
- **Embedding model:** converts documents and questions into vectors.
- **Vector store:** stores vectors and retrieves similar chunks.
- **Retriever:** provides search and may apply filters or reranking.
- **LLM:** generates the final response from question and context.
- **Prompt template:** controls grounding, missing-evidence behaviour, and output format.

19. Common failure modes

Symptom	Possible cause	Inspect or improve
Relevant text missing	Poor extraction, chunking, or embedding mismatch	Inspect extracted text, chunks, and search results
Wrong passage found	Ambiguous query or weak ranking	Try filters, hybrid search, query rewriting, reranking
Answer invents facts	Weak prompt or unsupported context	Require evidence and test unanswerable questions
Citation is wrong	Source metadata lost or mismatched	Carry document/page IDs through each stage

Security: retrieved documents are untrusted data, not instructions. A document could contain malicious text such as “ignore previous instructions.” Do not allow retrieved text to override system rules. Enforce permissions before retrieval and avoid exposing secrets.

ADVANCED RAG, EVALUATION & REVISION

20. Beyond basic RAG

Basic RAG retrieves once and passes the results to the LLM. Advanced RAG adds methods to improve relevance, coverage, context quality, and reliability. Add techniques when evaluation reveals a problem; complexity alone does not make a system better.

- **Query rewriting:** rewrite a vague question into a clearer search query.
- **Multi-query retrieval:** generate query variations and combine their results.
- **Reranking:** reorder candidate passages using a model that scores question–passage relevance.
- **Hybrid search:** combine dense semantic search with sparse keyword search.
- **Parent-child retrieval:** search small chunks but return a larger parent section for context.
- **Contextual compression:** remove irrelevant sentences before sending context to the LLM.
- **Corrective RAG:** assess evidence quality and search again or fall back when retrieval is weak.
- **Agentic RAG:** let an agent decide when and how to retrieve, potentially using multiple tools.

21. What is reranking?

A first-stage retriever may quickly return 20 candidate chunks. A reranker then evaluates the question against each candidate and reorders them by relevance. The application can send only the best few passages to the LLM. This combines broad initial search with more careful ranking.

22. RAG evaluation

Evaluate retrieval and generation separately. Build a test set of realistic questions, expected answers, and relevant source passages. Include answerable questions, ambiguous questions, and questions whose answers are absent from the knowledge base.

Metric	What it measures
Context precision	How many retrieved passages are actually relevant?
Context recall	Did retrieval find the evidence required to answer?
Faithfulness	Are answer claims supported by retrieved context?
Answer relevance	Does the response address the user's question?
Citation quality	Do citations point to the correct source and location?

23. Production considerations

- **Access control:** apply user/tenant permissions before returning chunks.
- **Freshness:** re-index changed documents and remove deleted content.
- **Observability:** log query, retrieved IDs, latency, model version, token usage, and errors safely.
- **Reliability:** handle empty retrieval, timeouts, malformed files, and provider rate limits.
- **Privacy:** minimise sensitive data, secure stored documents, and define retention.
- **Cost and latency:** monitor embedding, retrieval, reranking, and LLM token costs.

24. Quick interview revision

- **What is RAG?** Retrieve external evidence and supply it to an LLM to ground generation.
- **Why chunk?** Retrieve focused passages instead of entire documents.
- **What is an embedding?** A numerical vector representation of text meaning.
- **Dense vs sparse?** Dense uses semantic vectors; sparse uses keyword matching.
- **What is Top-K?** Number of candidate results returned by retrieval.
- **Why rerank?** Reorder candidates using a stronger relevance model.
- **How improve RAG?** Inspect failures, improve parsing/chunking, tune retrieval, rerank, prompt, and evaluate.

Recommended build order: (1) load one PDF, (2) extract and inspect text, (3) create chunks, (4) generate embeddings, (5) store and search vectors, (6) send retrieved context to an LLM, (7) return source references, and (8) test with 15–30 realistic questions. Get this basic pipeline working before adding agents or complex orchestration.